High-Performance Clifford Layers in C

TEAM 75

Simon Huber, Anirudhh Ramesh, Konstantinos Chasiotis, Mikkeline Elleby

ETH

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Clifford Algebra

- Operate on multivectors of scalars, vectors, planes, volumes, etc
- We work with basis vector that represent direction in space

- Uses geometric product with a main key rule:

$$e_i \times e_i = g_i$$

- This results in blades – a geometric object
- For a dimension d, we have 2^d blades which we define how they multiply with each other via a multiplication table. For 1d for instance it looks like this:

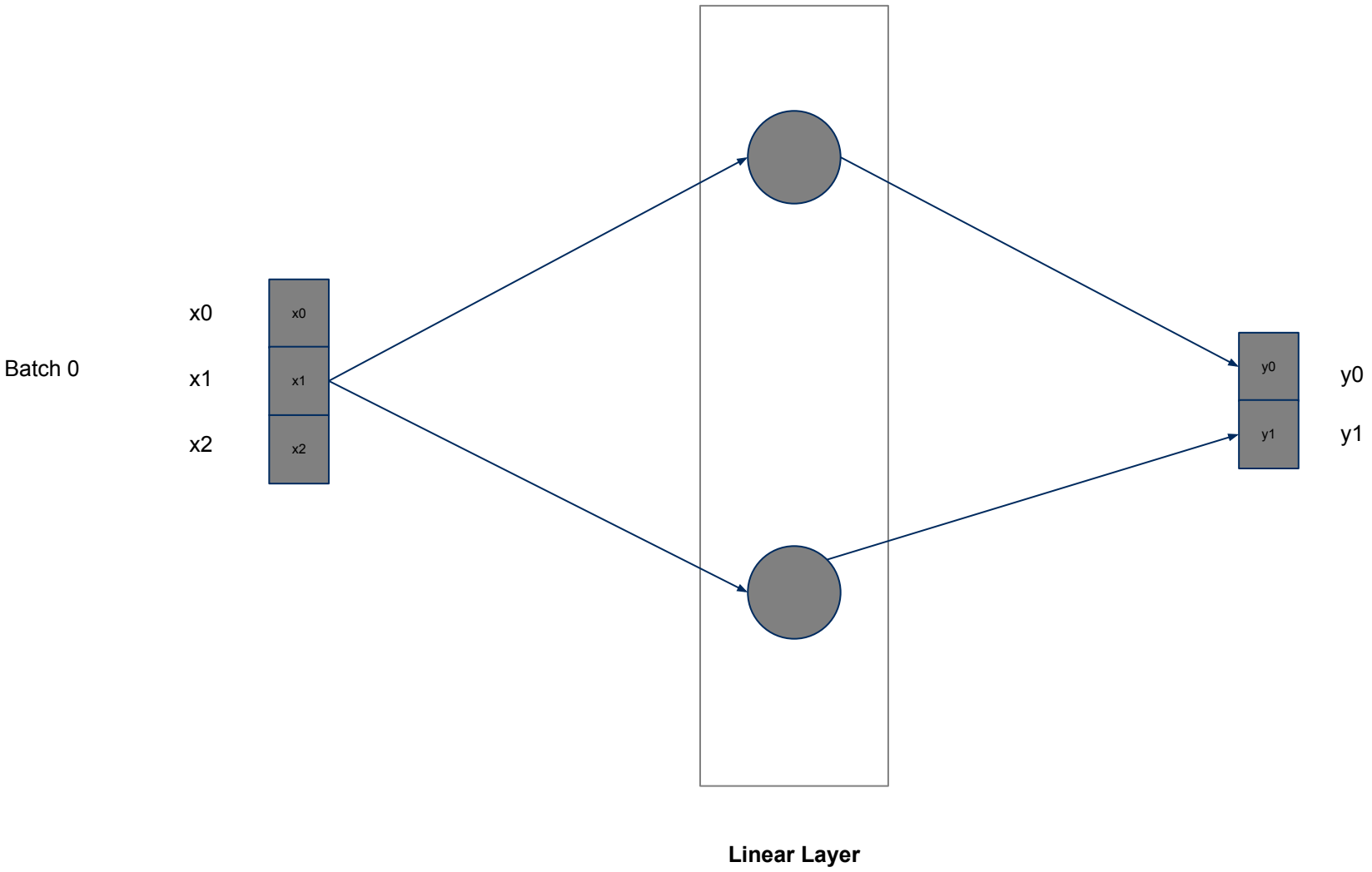
×	1	e_1
1	1	e_1
e_1	e_1	g_0

Reference Codebase

- Started with given codebase : GitHub - microsoft/cliffordlayers
- Created wrapper files for each c implementation to import the function into the original codebase
- Translated into c and focused on files for :
  - linear layer
  - multi-vector sigmoid linear units
  - group normalizations
- Extended the pytest for testing our implementation
  - Compared them with the python implementation

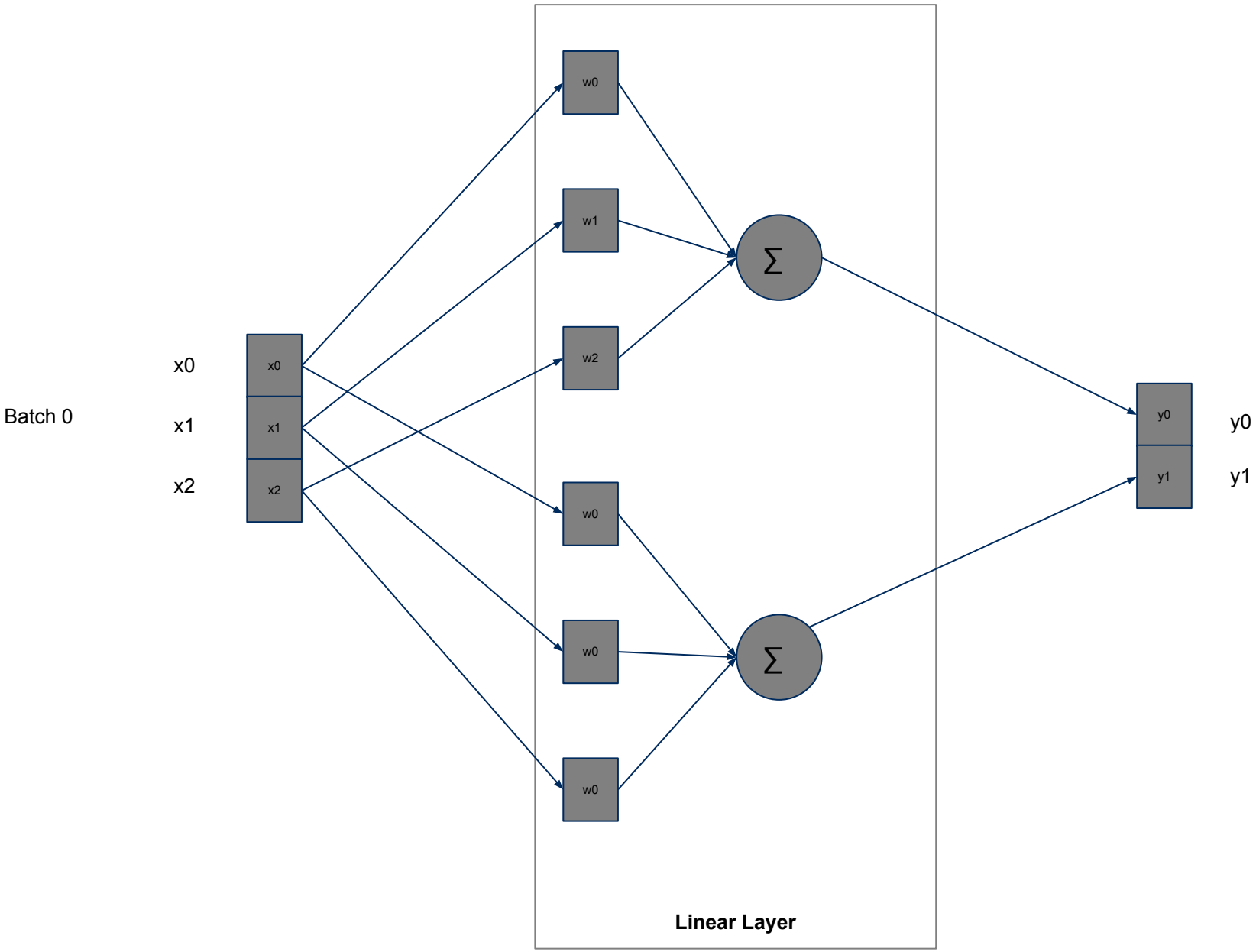
Layer: Linear Layer

Batch=1, channels_in = 3, channels_out = 2, dim=1



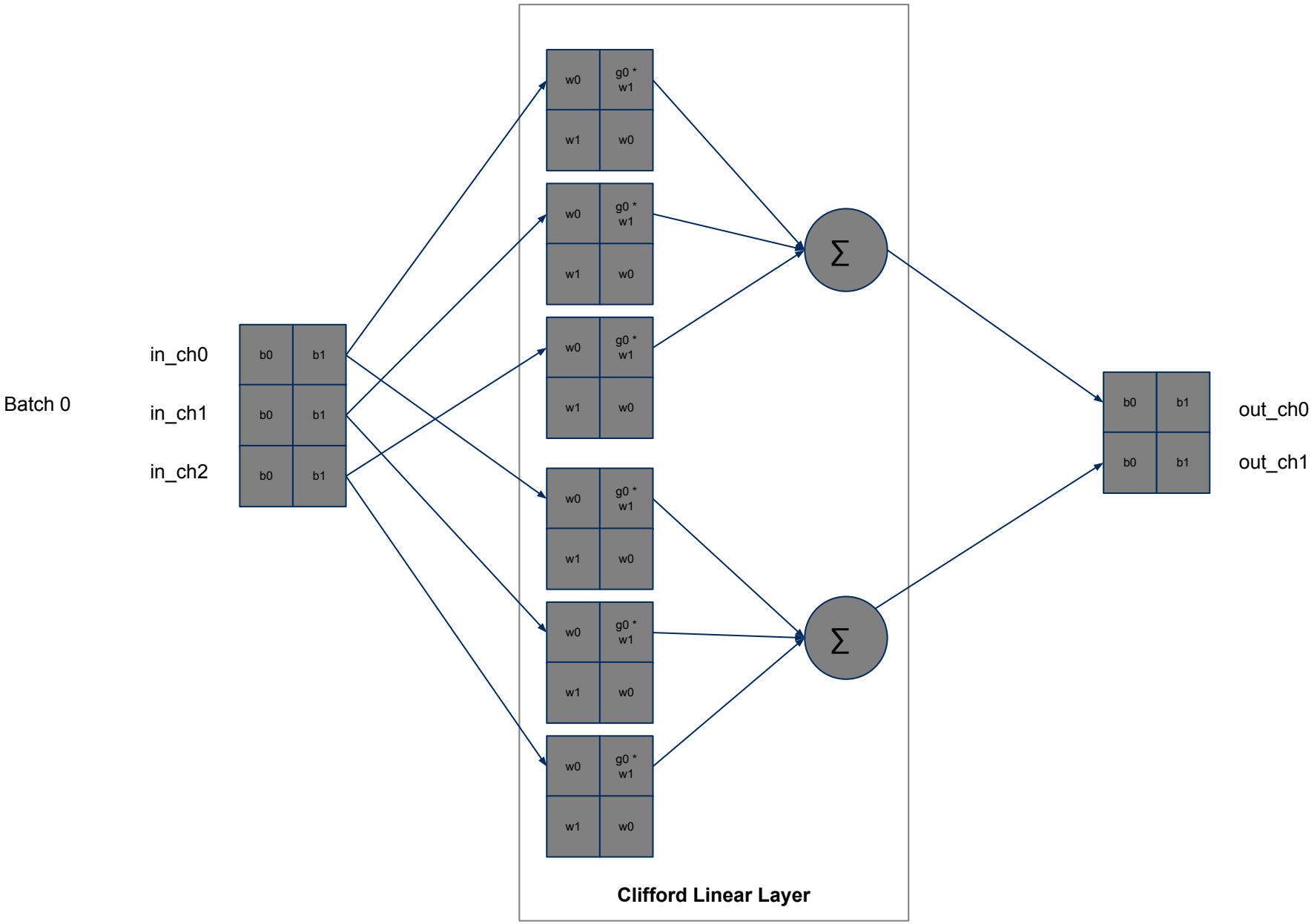
Layer: Linear Layer

Batch=1, channels_in = 3, channels_out = 2, dim=1



Layer: Clifford Linear Layer

Batch=1, channels_in = 3, channels_out = 2, dim=1

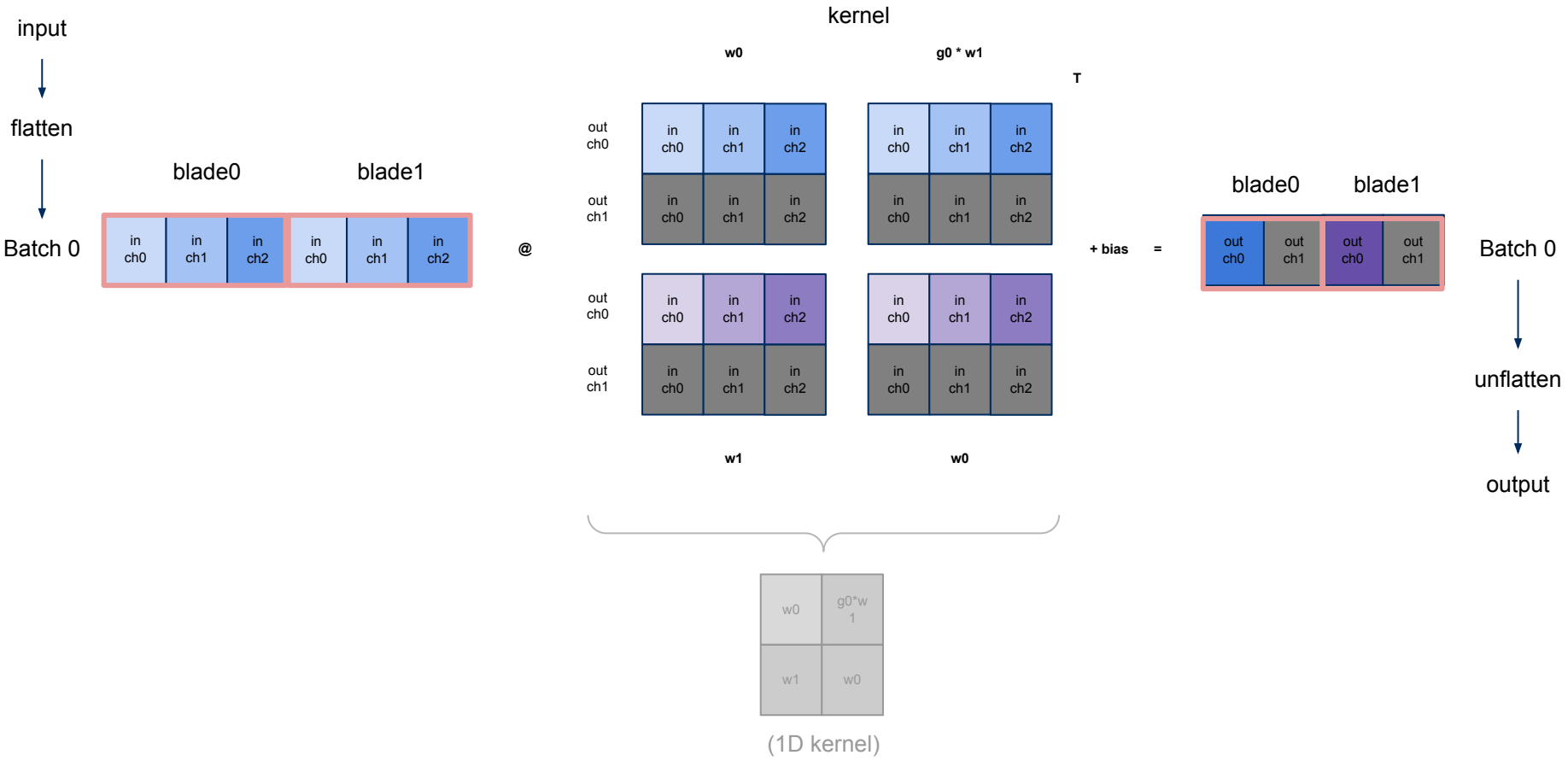


Batch=1, channels_in = 3, channels_out = 2, dim=1

Layer: Linear Layer

- **BASELINE**

Flattened input multiplied by kernels.



- **Optimization:**

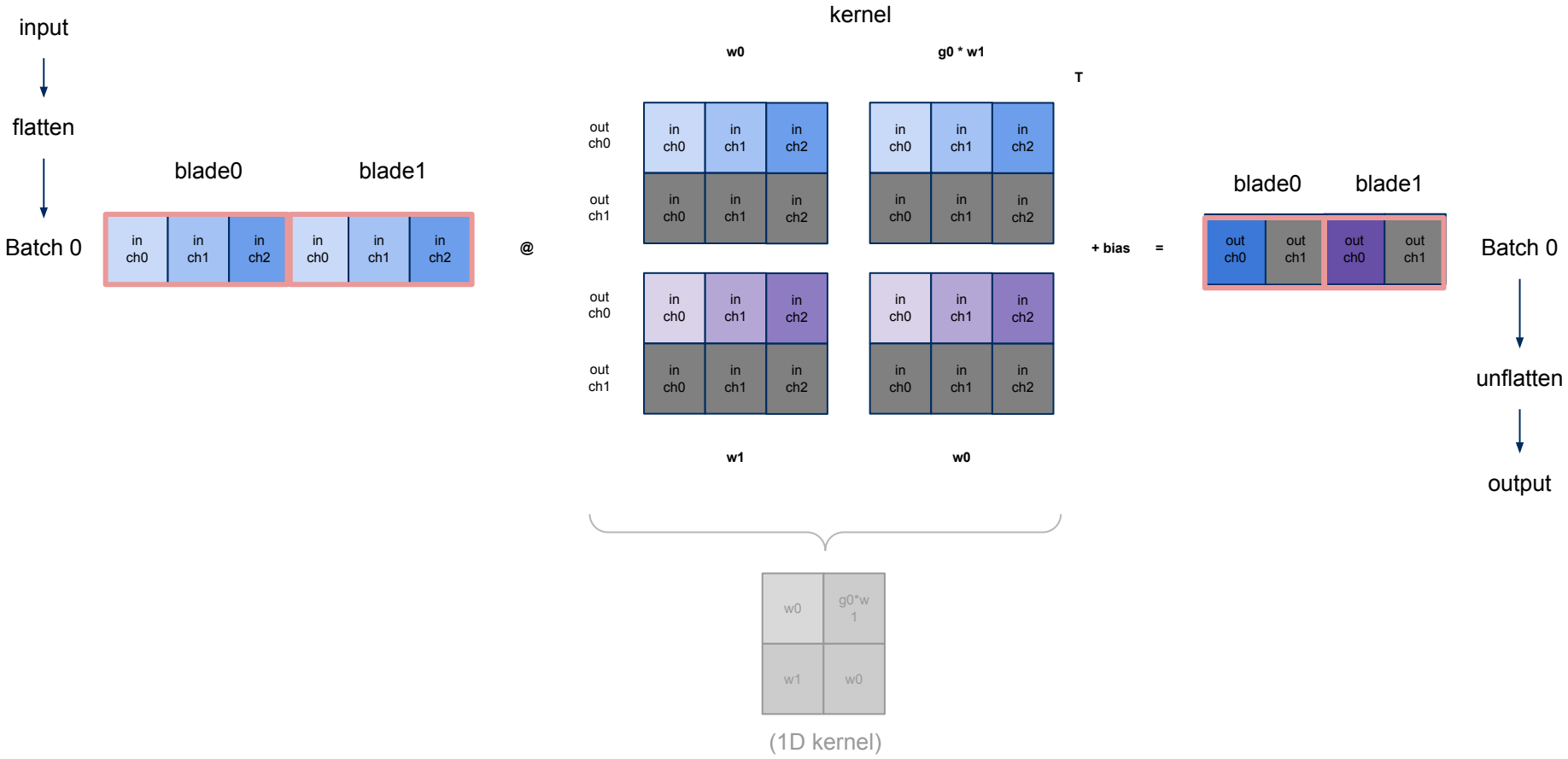
Cache the kernel (avoid reconstruction)

During kernel construction, pre-compute various multiple computations

AVX-2 vectorizations of the kernel multiplication, vector k-accumulators, FMA

=> ~3x overall scalar speed-up

=> ~8x overall vectorized speed-up

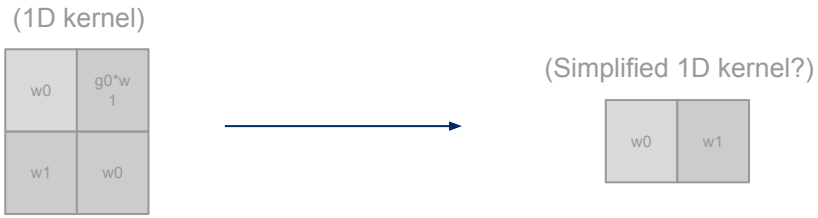


Layer: Linear Layer

- **Further Optimization ideas (in progress)**

Currently, the kernels look as follows:

1D:
[w0 g0*w1]
[w1 w0]

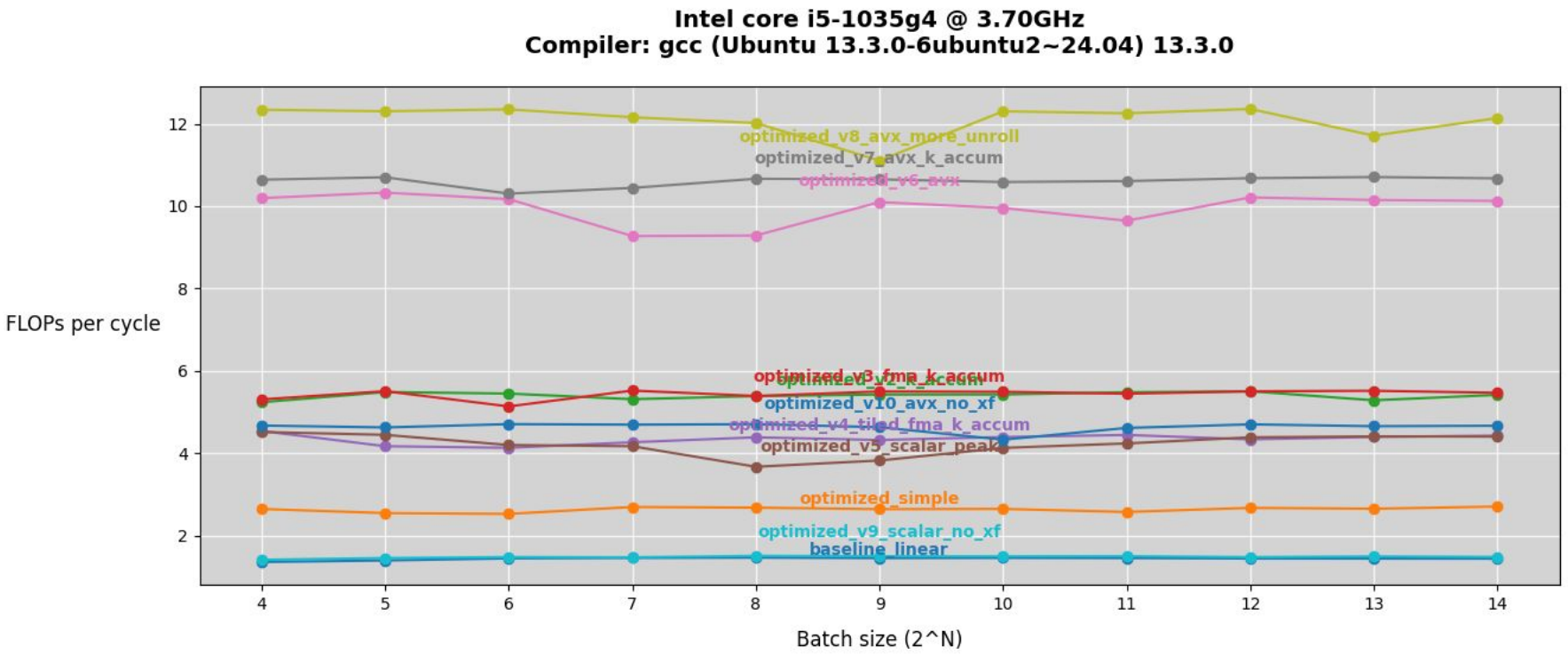


2d:
… (similar pattern)

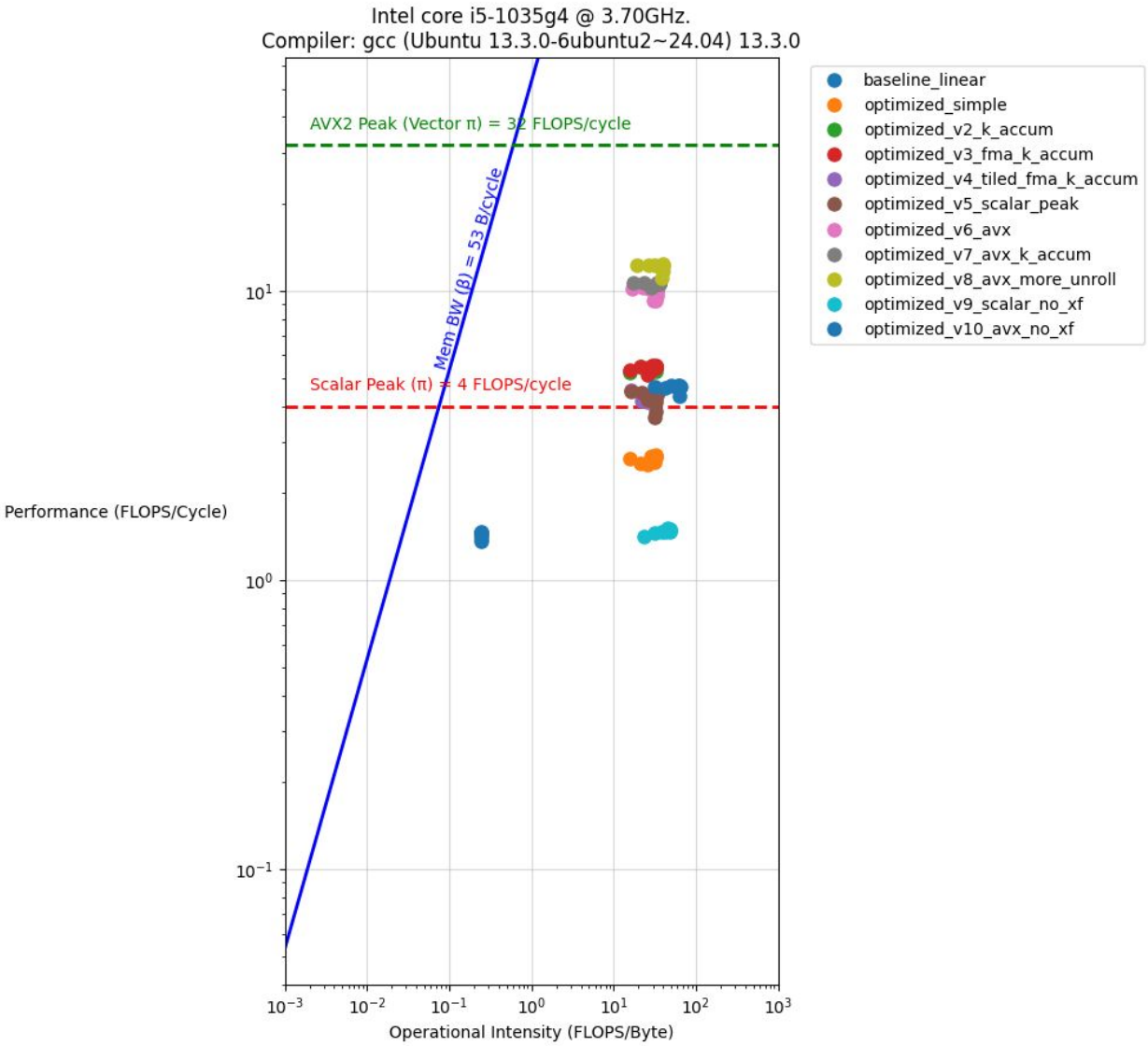
3d:
… (similar pattern)

Could we halve the size of the kernels (e.g. k_opt = [w0 g0*w1])?
Then, in the multiplications we can do e.g. x @ k_opt and k_opt @ x to construct the final output.
Another idea: Construct the kernels in a way that allows no flattening?
=> Results in a smaller kernel (i.e. less memory load cycles)

Layer: Linear Layer (Performance)

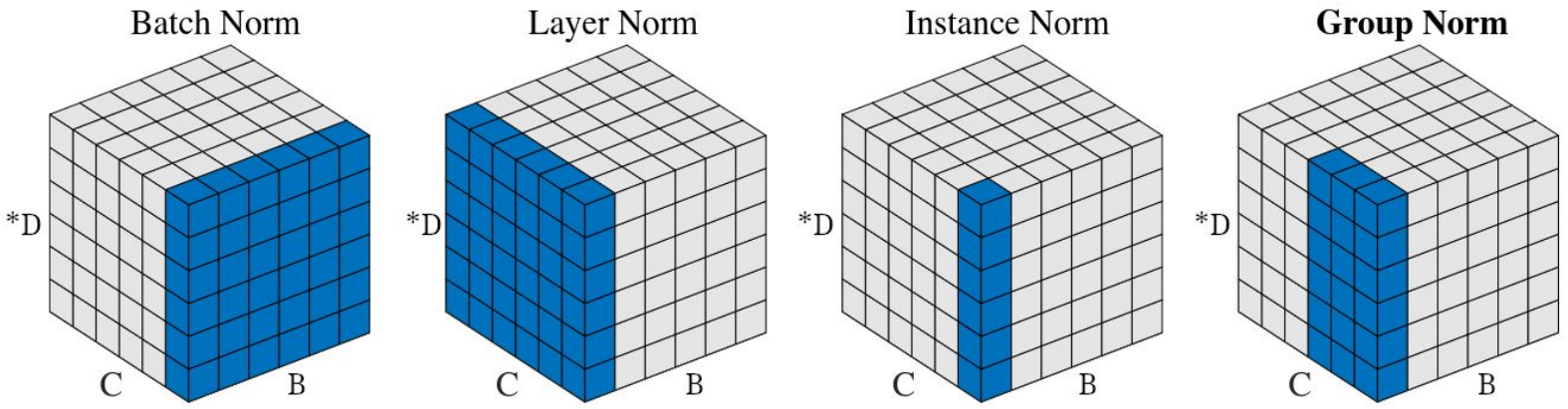


Layer: Linear Layer (Roofline)



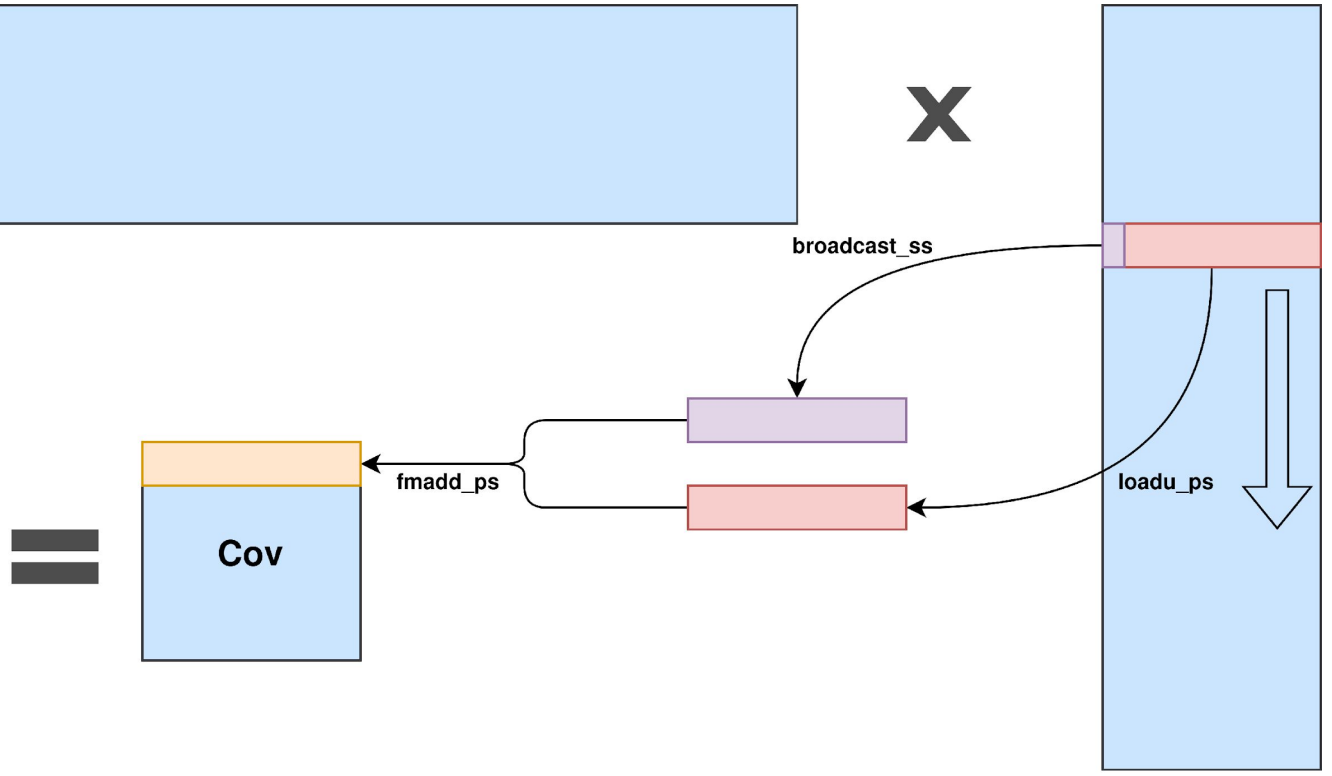
Layer: Group normalizations

- Normalizes across a group of channels
- Involves:
  - Calculating and subtracting means
  - Calculating covariance matrices
  - Finding maximums
  - Calculating Cholesky decompositions
  - Back-substitutions



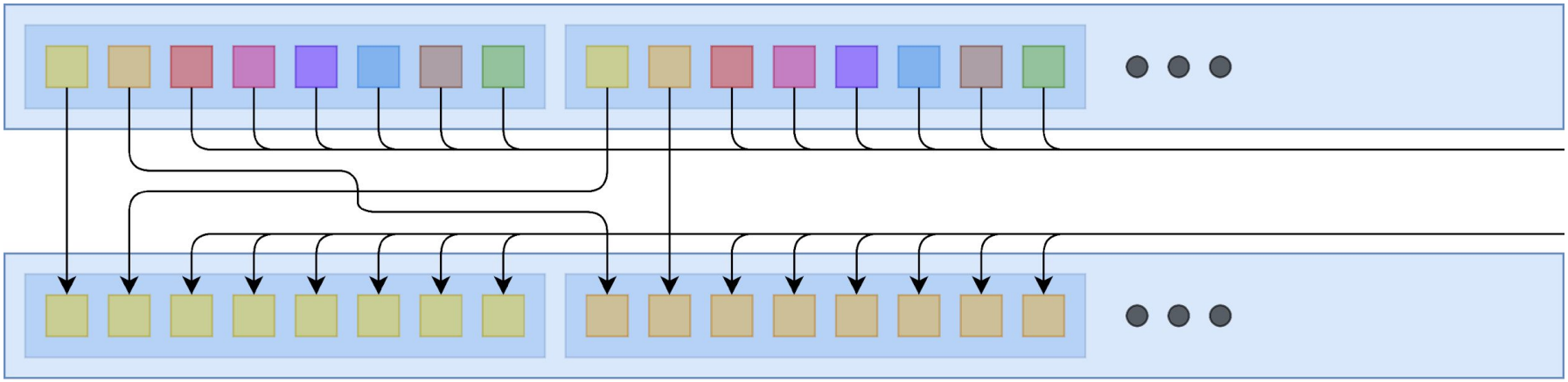
Groupnorm Covariance Optimization

- **BASELINE: Transpose into temporary array and do standard matrix multiply**
- **OPTIMIZATION: Vectorized direct calculation with AVX fmadd_ps**

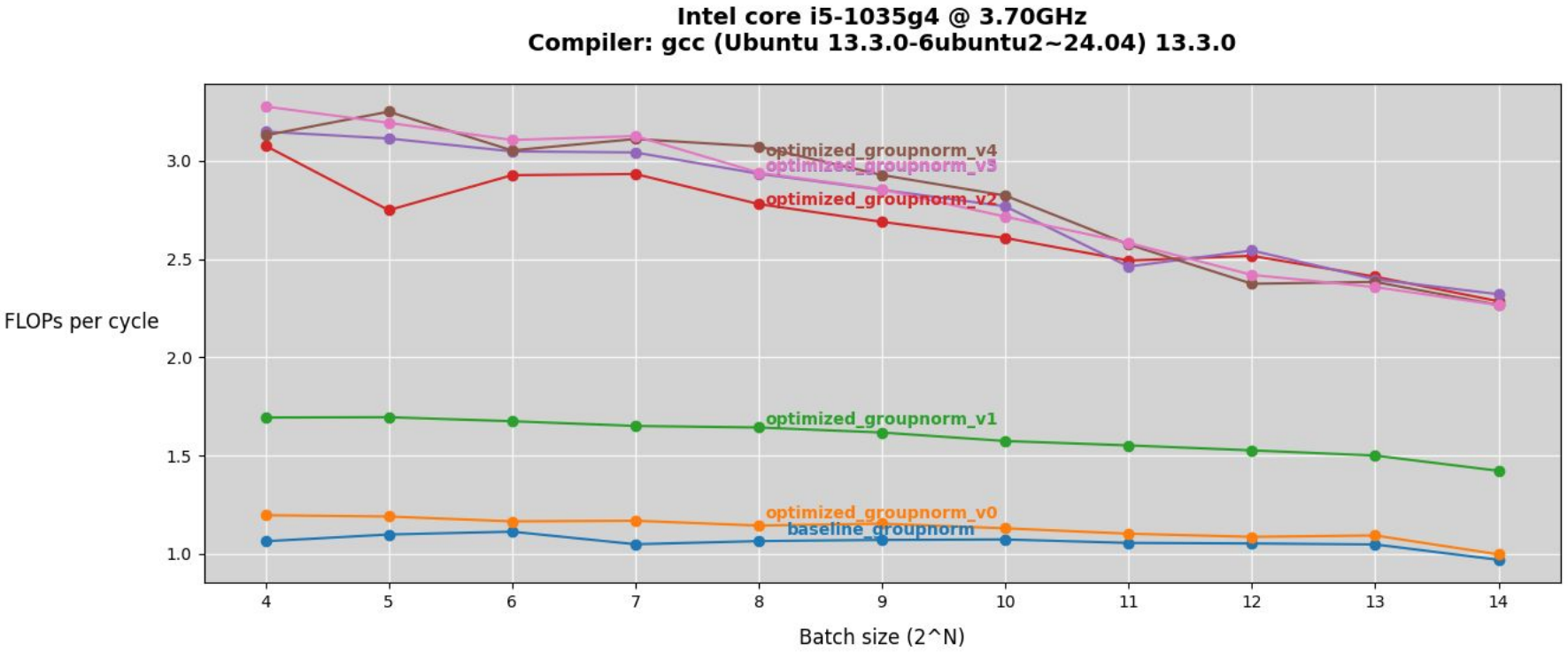


Groupnorm Backsubstitution Optimization

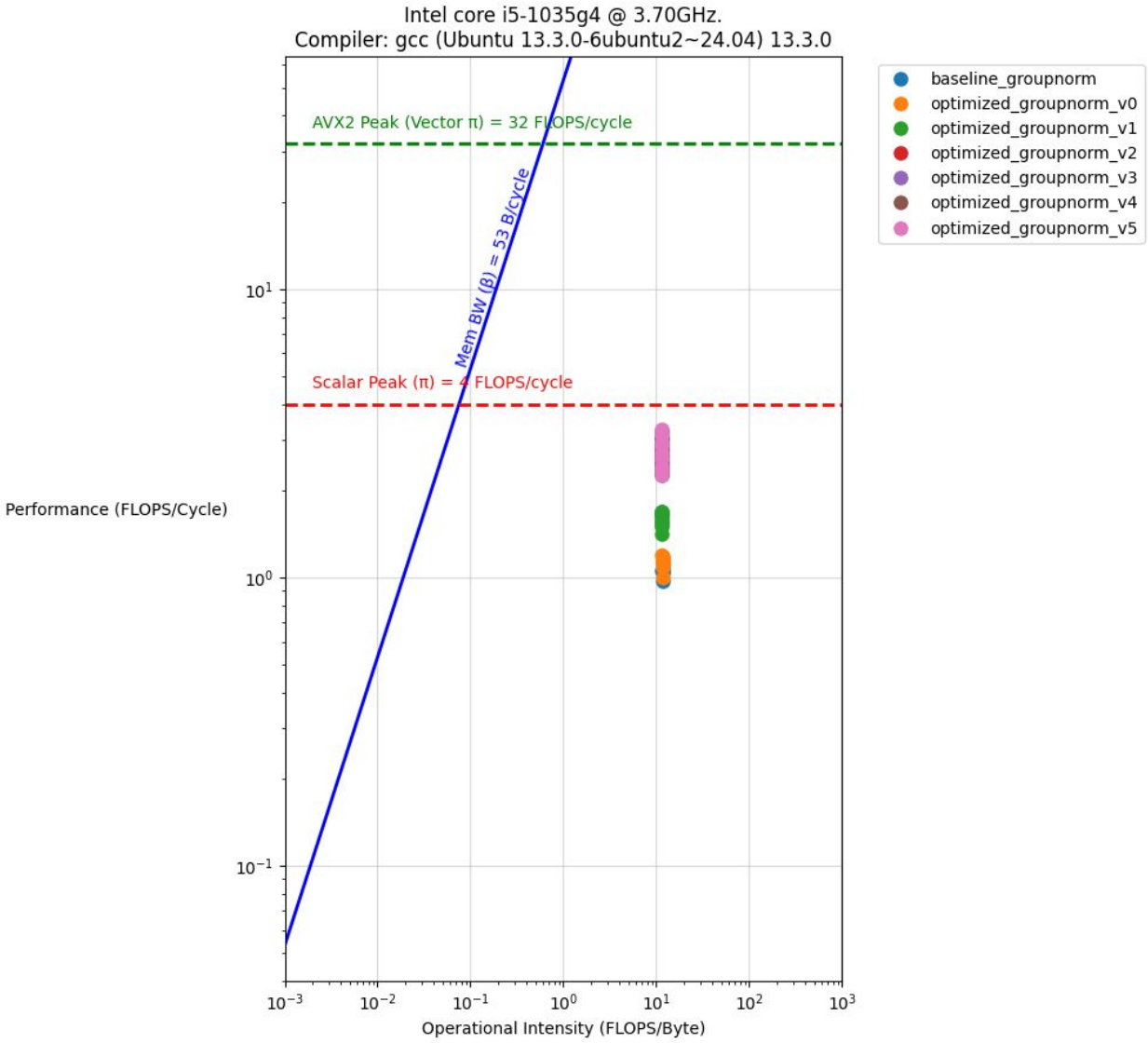
- Dependencies across iterations => hard to optimize single Back substitution
- BASELINE: One matrix and one vector at once
- OPTIMIZATION: One matrix and 8 vectors at once (AVX)
  - Needs memory layout restructuring



Groupnorm Performance Plots



Groupnorm Roofline



Multi-vector sigmoid linear units

1. **Pre-activation** z**:**

$$z = b_c + \sum_{j=0}^{I-1} W_{c,j} X_{b,c,p,j}$$

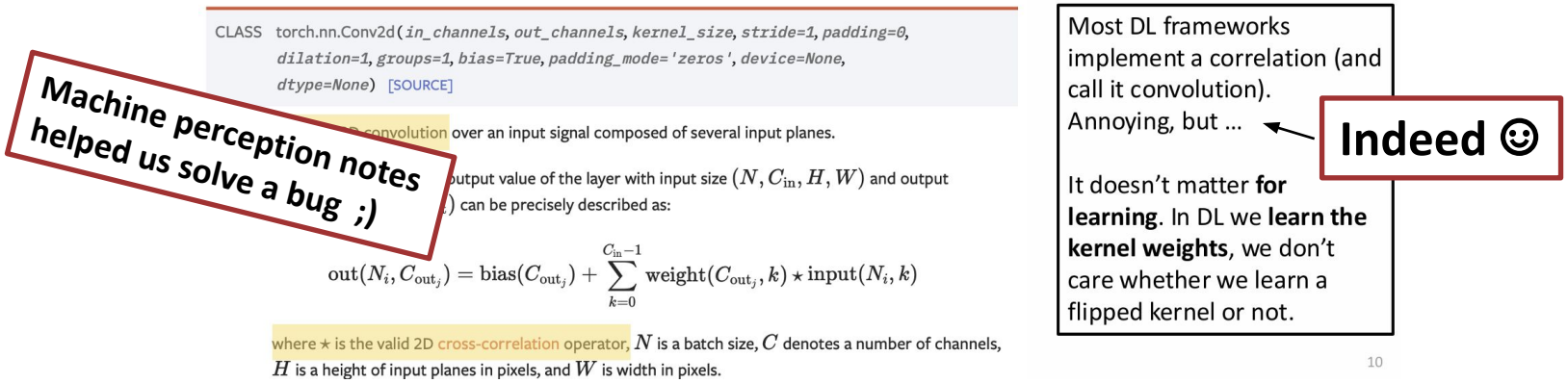
2. **Non-linearity** *gate*:

$$\text{gate} = \sigma(z) = \frac{1}{1+e^{-z}}$$

3. **Output** Y**:**

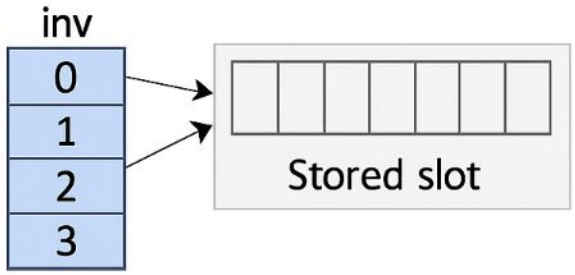
$$Y_{b,c,p,i} = X_{b,c,p,i} \times \text{gate}$$

- **Depth-wise 1×1 "convolution" → sigmoid → gate each blade component**



MV: Inverse mapping

- **Full blade space → Stored slots**
  - **We start with a large space of possible blade indices (0…all_blades–1), but only store a small subset (slots) per channel**



- **Build tiny lookup table:**

```
for (int b = 0; b < all_blades; ++b) inv[b] = -1;
for (int i = 0; i < slots; ++i) inv[input_blades[i]] = i;
```

Go from O(batch·channels·all_blade_space)

→ to O(all_blades) memory !

Note : all_blades = size of the full blade index range

MV: Fast Sigmoid via Lookup

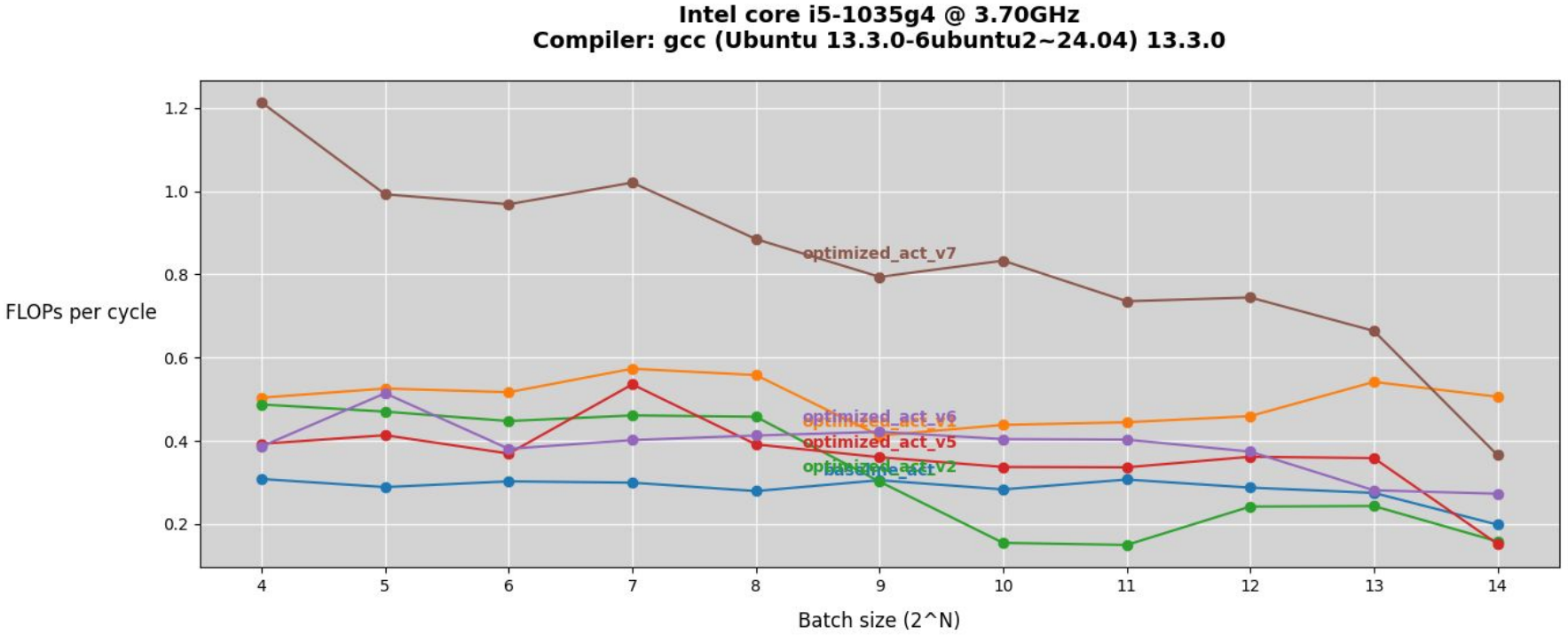
- Per-element 1/(1+expf(–z)) costs tens of cycles each
- Dominates our inner loop SO EVEN MORE EXPENSIVE !

  - Idea: Sample once by doing the computation
  - Lookup + interpolate:
    - *Clamp z to [−6,6]*

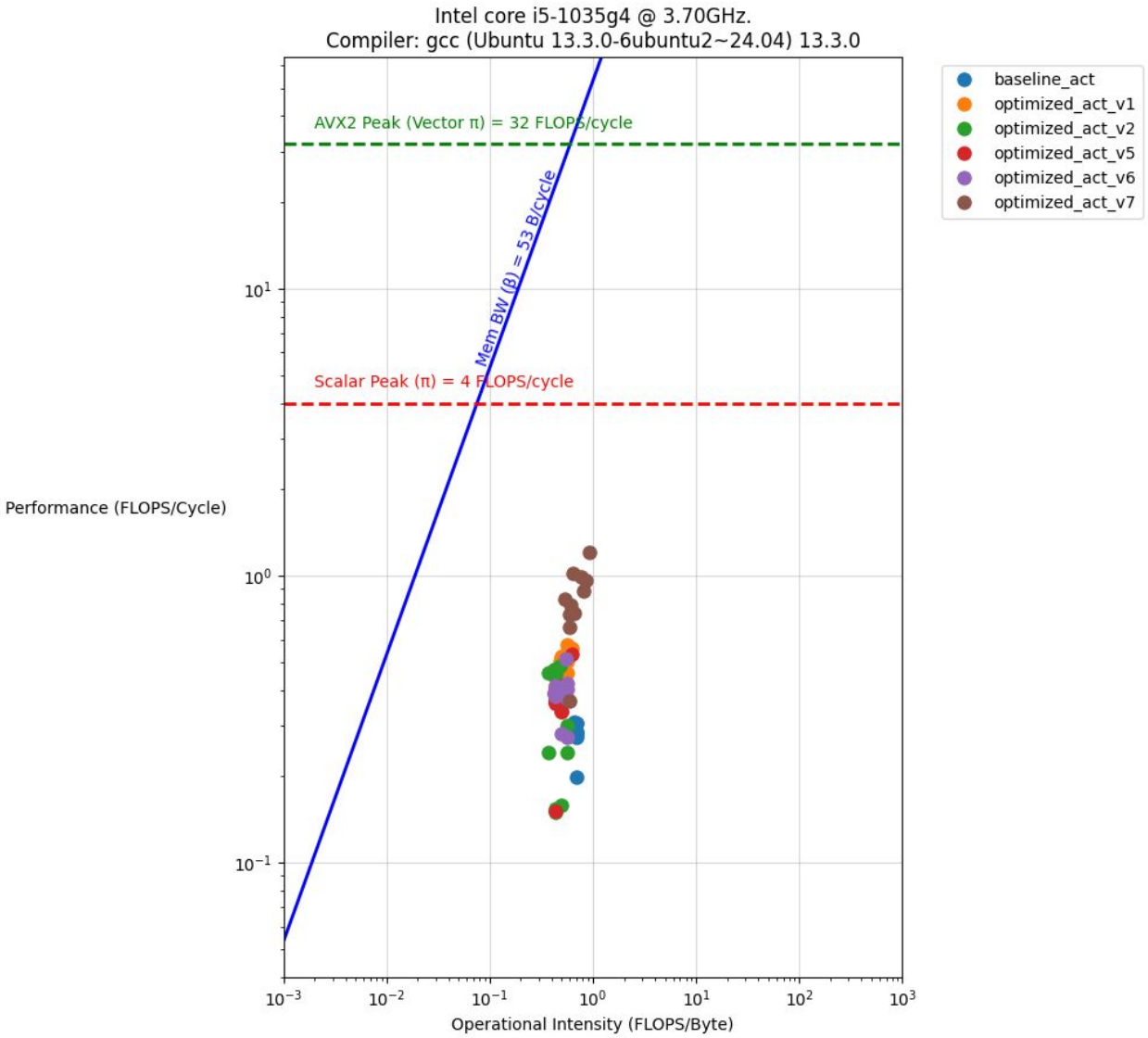
```
for (int i = 0; i < 512; ++i) {
  float x = -6.0f + 12.0f * i / 511.0f;
  sigmoid_table[i] = 1.0f / (1.0f + expf(-x));
}
```

- No expf() inside the hot loop—just two loads + three ops
- Still get ~1e-5 accuracy

Layer: Activation (Performance)



Layer: Activation (Roofline)



Thank you for listening !